AILang — Auditoría Completa del Repositorio

Arquitecto de Lenguajes / Ingeniero Senior de Compiladores

2026-10-10

AUDIT_REPORT.md — AILang

Repositorio auditado: `ailang-core-v7.2/` (extraído de `ailang-v5.0.0 (3).zip`) **Fecha de auditoría:** 2026-10-10 **Rol aplicado:** Arquitecto de lenguajes + Ingeniero senior de compiladores/IR/VM/runtime + Ingeniero de seguridad + Ingeniero de tooling + QA + Maintainer OSS **Alcance:** 73 archivos Python (10 413 LOC), 7 docs Markdown, 6 ejemplos `.ailang` / `.ail`, 14 archivos de tests. **Veredicto general:** ❌ **NO APTO para declararse 1.0.** El repositorio contiene componentes avanzados pero parcialmente divergentes, con duplicación estructural, claims falsos y divergencias interpreter↔VM documentadas.

0. Resumen ejecutivo

AILang v7.2 es un lenguaje funcional en su núcleo (variables, funciones, clases, herencia, try/catch, match, lambdas, closures, enumerados, registros, interfaces, generadores, async/await sintáctico) que se ejecuta correctamente contra **109 tests** bajo el intérprete tree-walking. Sin embargo, el repositorio presenta **cuatro problemas arquitectónicos bloqueantes** que impiden considerarlo coherente, verificable y seguro:

1. **Inversión de dependencias estructural.** El paquete modular `ailang/` no es autónomo: importa el monolito `ailang_core.py` (3528 LOC, 1/3 del código total) en tiempo de ejecución vía `runtime_bridge.execute_unified` y `stdlib/brain.bind_engine`. Sin el monolito, el runtime cognitivo no funciona.

2. **Dualidad de frontend.** Existen dos frontends paralelos: `ailang/compiler/` (DSL cognitivo, 347 LOC, v7.1 legacy) y `ailang/lang/` (lenguaje completo, 1156 LOC, normativo). Colisionan en el símbolo público `compile_source`. Solo `lang/` está cableado a CLI, tests y tooling.

3. **Interpreter ≠ VM.** La VM diverge del intérprete en **11 categorías de features**, incluyendo mutabilidad de closures (resultado incorrecto silencioso), generadores (error duro), spawn/channels (error duro), enum/interface/record (error duro), interpolated strings (error duro) y async/await (no-op en intérprete, error en VM). Esto viola la regla `Interpreter == VM` (FASE 12).

4. **Caos de versiones.** Coexisten **9 cadenas de versión distintas** (`1.0.0`, `2.0.0`, `5.0.0`, `v7.0`, `v7.1`, `7.1.0`, `v7.2`, `8.0.0`, `v8.1`, `v8.3`). Un test embebido en el monolito **falla garantizadamente** porque afirma `__version__ == "1.0.0"` cuando el código declara `__version__ = "2.0.0"`.

Adicionalmente, el repositorio contiene **afirmaciones falsas documentadas**: PostgreSQL backend con `return []` en todos sus métodos, GPU/NPU/TPU backends que siempre delegan a CPU, debugger que registra breakpoints pero no pausa, `execute_unified` que ignora silenciosamente `uir.agent` y `uir.task`, y un `re_mod.search` que lanza `NameError` por usar `null` (no definido en Python).

Deuda técnica total estimada: P0 = 11 tareas bloqueantes · P1 = 14 tareas de estabilidad · P2 = 16 tareas de tooling · P3 = 9 tareas de optimización · P4 = 7 áreas avanzadas.

---

1. Estado actual del repositorio

1.1 Estructura encontrada

```
ailang-core-v7.2/
├── ailang_core.py                  ← MONOLITO 3528 LOC (1/3 del código)
├── ailang/                         ← Paquete modular normativo
│   ├── __init__.py                 ← __version__ = "5.0.0"
│   ├── __main__.py                 ← CLI (171 LOC, 0% coverage)
│   ├── lang/                       ← Frontend completo (lexer/parser/ast/typecheck/interp/vm)
│   │   ├── lexer.py                ← 22 tokens, 32 keywords
│   │   ├── parser.py               ← 559 LOC, recursive-descent
│   │   ├── ast_nodes.py            ← 30 AST dataclasses
│   │   ├── typecheck.py            ← Gradual, env+narrowing (235 LOC)
│   │   ├── interpreter.py          ← Tree-walking (843 LOC)
│   │   ├── vm.py                   ← Stack VM (699 LOC) — DIVERGENTE
│   │   ├── bytecode_io.py          ← .ailc loader/saver
│   │   ├── errors.py
│   │   └── stdlib/                 ← 13 módulos (io/math/strings/json/re/time/http/path/sys/random/
listops/test/brain)
│   ├── compiler/                   ← Frontend COGNITIVO legacy v7.1 (347 LOC) — DUPLICADO
│   │   ├── lexer.py / parser.py / semantic.py / pipeline.py
│   ├── ir/                         ← 5 IRs tipados (sin pipeline HIR→MIR→AIR)
│   │   ├── unified.py  cognitive.py  agent.py  task.py  security.py
│   ├── storage/                    ← SQLite (real) + PostgreSQL (FAKE)
│   ├── hardware/                   ← CPU (real) + GPU/NPU/TPU (stubs)
│   ├── lsp.py / dap.py / debugger.py / profiler.py / fmt.py / lint.py
│   ├── pkg.py / project.py / docgen.py / watch.py / bench.py / coverage_tool.py
│   ├── runtime_bridge.py           ← Puente al monolito
│   └── agent_runtime.py            ← Agent/Task runtimes (stubs)
├── tests/                          ← 14 archivos, 109 tests
├── docs/                           ← 5 docs Markdown (alineación parcial)
├── examples/                       ← 5 .ailang + 1 .ail (roto bajo CLI estándar)
├── pyproject.toml                  ← version = "7.1.0" (discordante)
├── README.md                       ← "AILang 5.0.0"
├── CHANGELOG.md                    ← top entry "5.0.0 — Nivel E"
└── LICENSE
```

1.2 Métricas generales

Métrica	Valor
LOC Python total	10 413
Archivos Python	73
Archivos Markdown	7
Archivos de tests	14 (109 tests)
Tests embebidos en monolito	44 (1 fallando)
Cobertura de línea global	72 %
Módulos con 0 % cobertura	4 (`__main__.py` , `lsp.py` , `dap.py` , `watch.py`)
Versiones distintas detectadas	≥ 9
TODOs / FIXMEs / XXX / HACK explícitos	0 (toda la deuda está silenciada como `NotImplementedError` o `return []`)
`NotImplementedError` en producción	2 (PostgreSQL backend)
Stubs / mocks / fake impls catalogados	13

1.3 Pipeline de ejecución real (lo que el código hace, no lo que dice hacer)

```
Source (.ailang)
   ↓
ailang/lang/lexer.py             ← Tokeniza 22 token-types, 32 keywords
   ↓
ailang/lang/parser.py            ← Recursive-descent → AST (30 dataclasses)
   ↓
ailang/lang/typecheck.py         ← Gradual, optional strict
   ↓
┌── Interpreter (default)  →  ailang/lang/interpreter.py  ← Tree-walk, 843 LOC
│       └── brain.* calls  →  ailang/lang/stdlib/brain.py  →  runtime_bridge  →
ailang_core.CognitiveBrainEngine
│
└── VM (--vm o AILANG_VM=1) →  ailang/lang/vm.py  ← Stack VM, 699 LOC
        ├── Diverge en 11 categorías vs intérprete
        └── No registra builtins `type`, `isinstance`, `implements`, `channel`, `brain_*`
```

No existe HIR, ni MIR, ni AIR como representaciones intermedias con semántica de lowering. Los archivos `ailang/ir/*.py` son **manifests JSON-versionados con digest SHA-256** (esencialmente dataclasses congeladas) — útiles como metadatos pero **no son un compilador**.

---

2. Arquitectura encontrada vs arquitectura objetivo (FASE 2)

2.1 Arquitectura objetivo demandada por el prompt maestro

```
Source → Lexer → Parser → AST → Name/Scope → TypeChecker → Effect → Capability
       → TypedAST → HIR → MIR → AIR → { Interpreter | Bytecode | VM | Native | WASM }
```

2.2 Arquitectura real encontrada

```
Source → Lexer (lang/) → Parser (lang/) → AST (lang/ast_nodes.py)
       → Typecheck (lang/typecheck.py) — opcional, no obligatorio
       → Interpreter (lang/interpreter.py) ← tree-walk directo sobre AST
       └── VM (lang/vm.py) ← compila AST a opcodes ad-hoc, NO via IR

PARALELO:
Source → Lexer (compiler/) → Parser (compiler/) → semantic.py → CognitiveIR/UnifiedIR
       → runtime_bridge.execute_unified(uir, engine=ailang_core.CognitiveBrainEngine)

PARALELO:
ailang_core.py contiene SU PROPIO CognitiveCompiler (línea 372) + AILangCompiler (línea 552)
+ CognitiveBrainStore (línea 2243) + CognitiveBrainEngine (línea 2724)
+ CLI (línea 2789) + 44 tests embebidos (línea 3063)
```

2.3 Brechas arquitectónicas

Componente demandado	Existe?	Comentario
Lexer único	❌	Dos: `compiler/lexer.py` (DSL) y `lang/lexer.py` (lenguaje).
Parser único	❌	Dos: `compiler/parser.py` y `lang/parser.py`.
AST estable	⚠️	`lang/ast_nodes.py` (30 nodos) es normativo. El `compiler/` usa 2 nodos propios.
Name/Scope resolution	⚠️	Implícito en `Env` dentro del intérprete; no es un pass separado.
Type checker	⚠️	Existe (`typecheck.py`, 235 LOC) pero **no es obligatorio** en el pipeline de run.
Effect checker	❌	No existe.
Capability/Security checker	⚠️	`ir/security.py` define `Capability` y `authorize()` pero **no se invoca desde el runtime**. Solo se valida en `UnifiedIR.validate()`.
Typed/Resolved AST	❌	No existe; el intérprete opera sobre el AST crudo.
HIR	❌	No existe.
MIR	❌	No existe.
AIR	❌	No existe. El "AIR" mencionado en docs no aparece ni una vez en el código.
Interpreter	✅	Tree-walking, 843 LOC, funcional para 109 tests.
Bytecode compiler	⚠️	`bytecode_io.py` serializa/deserializa pero el formato está acoplado al intérprete, no es portable.
VM	⚠️	699 LOC, diverge en 11 categorías.
Native backend (LLVM)	❌	No existe.
WASM backend	❌	No existe.
Runtime separado del frontend	❌	Runtime embebido en `ailang/lang/`.

Conclusión FASE 2: La arquitectura encontrada es una **versión temprana del nivel "AST → Interpreter/VM"**, no el pipeline HIR→MIR→AIR que demanda el prompt. Las capas cognitivas **sí son**

compiladores paralelos (CognitiveCompiler, AILangCompiler, CognitiveBrainStore, CognitiveBrainEngine), violando explícitamente FASE 2 y FASE 9.

---

3. Funcionalidades existentes (lo que SÍ funciona)

3.1 Lenguaje — intérprete

Feature	Interpreter	Test que la ejercita
`let`, `const` (const sin enforce en runtime)	✅	`test_core_lang.py::test_let_and_const`, `test_nivel_c.py::test_const_immutability`
`if / else`, `while`, `for x in iter`	✅	`test_core_lang.py` (6 casos)
`fn` + `return`, lambdas, recursion	✅	`test_core_lang.py::test_fn_and_return`, `test_oop_closures.py::test_lambda_*`
Mutable closures	✅	`test_oop_closures.py::test_mutable_closure`
`class`, `extends`, `super`, `self`, polimorfismo	✅	`test_oop_closures.py` (4 casos), `test_v8_final.py::test_polymorphism`
`try / catch / throw`	✅	`test_modules_stdlib.py` (3 casos), `test_vm_try_catch.py`
`match` (literal + binding)	✅	`test_nivel_c.py::test_match`
`break / continue`	✅	`test_v8_final.py::test_break_continue`
`enum`	✅	`test_nivel_e.py::test_enum`
`record`	✅	`test_nivel_e.py::test_record`
`interface` + `implements()`	✅	`test_nivel_e.py::test_interface_implements`
Generadores (`yield`)	✅	`test_nivel_e.py::test_generator`
`channel()` + `send` / `recv`	✅	`test_nivel_e.py::test_channel`
`spawn`	✅	`test_nivel_e.py` (vía `threading.Thread`)

Feature	Interpreter	Test que la ejercita
Interpolated strings `"${x}"`	✅	`test_nivel_e.py::test_interpolation`
`async fn` / `await`	⚠️ sintaxis parseable, **await es no-op**	`test_nivel_d.py::test_async_await` (solo verifica que no crashea)
`import` (builtins + módulos `.ailang`)	✅	`test_modules_stdlib.py::test_import_*`, `test_vm_class_import.py`
`purge_after(Nms)` directive	✅	`test_conformance.py::test_C10_purge_after`

3.2 VM — subconjunto core

Feature	VM
Aritmética, `let`, `fn`, `return`, `if`, `while`, `class`, `try/catch`, `import`, `super`, polimorfismo, `break` / `continue`	✅
Todo lo demás (ver §5 divergencias)	❌

3.3 Stdlib — 12 de 13 módulos reales

`io`, `math`, `strings`, `json`, `time`, `http` (urllib real), `path`, `sys`, `random`, `listops`, `test_mod`, `re` (con bug en `search`). El módulo `brain` es un puente de delegación que requiere un engine externo.

3.4 Storage — SQLite real

`SQLiteStorage` implementa CRUD completo sobre `docs` + `chunks`, con búsqueda lexical (bag-of-words) y búsqueda vectorial (cosine con NumPy). Es real y testeado.

3.5 Tooling que funciona end-to-end

`run`, `--repl`, `--vm`, `--strict`, `--version`, `--lsp` (parcial), `--dap` (parcial, sin pausa real), `--test`, `pkg init/install/list/run`, `fmt`, `lint`, `profile` (limitado), `compile`, `runbc`, `coverage` (limitado), `watch`, `project init/run/show`, `bench`, `docgen`.

---

4. Funcionalidades incompletas o falsas

4.1 Catalogación de implementaciones falsas / stubs / mocks

#	Ubicación	Categoría	Severidad	Evidencia
F1	`storage/postgres_backend.py:68-91`	**FAKE**	🔴 P0	`upsert_document` y `replace_chunks` hacen `raise NotImplementedError`; `list_documents`, `list_chunks`, `search_lexical`, `search_vector` devuelven `[]`. El SQL real está como comentario en la línea 91.
F2	`hardware/accel.py:1-69`	**FAKE**	🔴 P0	`GPUBackend`, `NPUBackend`, `TPUBackend` heredan de `_AccelStub` cuyos `to_device` / `matmul` / `vector_score` siempre delegan a `CPUBackend`. El docstring del módulo admite "Stubs".
F3	`agent_runtime.py:42-43`	**STUB**	🟠 P1	`AgentStepKind.ACT` → `ctx.memory[step.name] = step.args` (almacena args, no ejecuta nada).
F4	`agent_runtime.py:40-41`	**MOCK**	🟠 P1	`THINK` fallback → `{"thought": query, "mode": "echo"}` cuando no hay brain.
F5	`agent_runtime.py:52-56`	**STUB**	🟠 P1	`DECIDE` → `ctx.memory[step.name] = {"decision": step.args.get("choice", "continue")}` — solo relabel.
F6	`agent_runtime.py:60-83`	**FAKE**	🔴 P0	`TaskRuntime.run` marca cada tarea `PENDING → DONE` sin invocar payload ni retornar resultados. La lista `results` contiene IDs, no outputs.

#	Ubicación	Categoría	Severidad	Evidencia
F7	`runtime_bridge.py:43-47`	**OMISIÓN**	🔴 P0	`execute_unified` valida `uir` pero solo despacha `uir.cognitive`. `uir.agent` y `uir.task` son ignorados silenciosamente.
F8	`lang/interpreter.py:666-670`	**NO-OP**	🔴 P0	`A.Await` → `return val` — await no hace nada; no hay event loop.
F9	`lang/vm.py:294`	**ERROR DURO**	🔴 P0	`raise AILangRuntimeError("VM no soporta {EnumDef,InterfaceDef,RecordDef,Spawn,Yield}")`
F10	`lang/vm.py:334`	**ERROR DURO**	🔴 P0	`raise AILangRuntimeError("expresión no compilable: {InterpolatedString,Await}")`
F11	`lang/vm.py:568-573`	**BUG SILENCIOSO**	🔴 P0	`MAKE_FUNCTION` captura `clos = dict(self.globals); clos.update(names_local)` (snapshot). Mutaciones posteriores a closures capturados no se reflejan. Test `counter()` devuelve 1 en VM donde el intérprete devuelve 2.
F12	`lang/vm.py:680-690`	**OMISIÓN**	🟠 P1	`run_vm` solo registra `print/len/str/int/float/range/true/false/null`. Faltan: `type`, `isinstance`, `implements`, `channel`, `brain_*`.
F13	`debugger.py:73`	**BUG**	🔴 P0	El body del bloque `if self.state.paused and line in self.breakpoints:` es `pass`. El debugger **nunca pausa**.
F14	`lang/stdlib/re_mod.py:6`	**BUG**	🔴 P0	`return null if False else None` — `null` no está definido en Python → `NameError` en cada búsqueda regex fallida.
F15	`ailang_core.py:2908, 2917`	**BUG**	🔴 P0	CLI `list outbox.db` y `purge-dead outbox.db` llaman a `PersistentOutbox.list_pending()` y `purge_dead()` — **métodos inexistentes** → `AttributeError`.
F16	`ailang_core.py:3429`	**TEST ROMO**	🔴 P0	`self.assertEqual(__version__, "1.0.0")` cuando `__version__ = "2.0.0"` (línea 118). El test embebido del monolito siempre falla.
F17	`ailang_core.py:605-616`	**API SIN CONSUMIDOR**	🟡 P2	`sign_bytecode` y `verify_bytecode_signature` existen pero `AILangCompiler.compile()` y `decode_bytecode()` no los invocan. Las firmas nunca se producen ni validan.

#	Ubicación	Categoría	Severidad	Evidencia
F18	`ailang_core.py:1453-1455`	**DOWNGRADE SILENCIOSO**	🟠 P1	`_try_fernet()` devuelve `("cryptography no instalado; outbox sin cifrar")` en ImportError. `PersistentOutbox` solo exige cifrado si `require_encryption=True`. Caso contrario, almacena payloads en cleartext a pesar del claim "Cifrado del outbox obligatorio si hay `secret`".
F19	`lang/interpreter.py:380-398` + `lang/vm.py:694-698`	**DIVERGENCIA**	🟠 P1	El intérprete registra `brain_*` solo si se pasa `brain_engine=`; la VM hace `vm.globals["brain"] = vm.load_module("brain")` envuelto en `except Exception: pass` — cualquier fallo se traga.
F20	`ir/unified.py` claim "Unified Cognitive + Security + Agent + Task"	**MARKETING FALSO**	🟠 P1	En runtime solo se despacha `cognitive`. Agent y Task son muertos.

4.2 Módulos marcados como ausentes

Módulo / Feature demandado	Estado
Tensor (FASE 33)	❌ Ausente. Solo existe `TensorRef` dataclass (4 campos).
HAL (FASE 34)	⚠️ Skeleton ABC con 5 métodos. Sin memory pools, streams, events, kernels.
Cryptography (FASE 35)	❌ Ausente. Ningún módulo de crypto en stdlib.
Native/LLVM (FASE 31)	❌ Ausente.
WASM (FASE 32)	❌ Ausente.
Effect checker (FASE 11)	❌ Ausente.
Capability enforcement en runtime (FASE 11)	⚠️ Existe `SecurityIR.authorize()` pero **no se invoca en runtime**.

5. Divergencias Interpreter ↔ VM (FASE 12)

#	Feature	Interpreter	VM	Divergencia
D1	Mutable closures	✅ correcto	❌ snapshot → resultado incorrecto silencioso	🔴 Bloqueante
D2	`yield` / generadores	✅ `GeneratorValue` + `YieldSignal`	❌ `raise`	🔴 Bloqueante
D3	`spawn`	✅ `threading.Thread`	❌ `raise`	🔴 Bloqueante
D4	`channel()` / `send` / `recv`	✅ `ChannelValue(queue.Queue)`	❌ no compilado	🔴 Bloqueante
D5	`enum`	✅ `EnumType/EnumMember`	❌ `raise`	🔴 Bloqueante
D6	`interface`	✅ `InterfaceType`	❌ `raise`	🔴 Bloqueante
D7	`record`	✅ `RecordType/RecordInstance`	❌ `raise`	🔴 Bloqueante
D8	Interpolated strings `"${x}"`	✅ partes + join	❌ `raise`	🔴 Bloqueante
D9	`await`	⚠️ no-op (`return val`)	❌ `raise`	🔴 Ambos rotos
D10	`async fn`	⚠️ flag almacenado, nunca honrado	❌ SyntaxError en muchas posiciones	🔴 Ambos rotos
D11	`const` immutabilidad	✅ enforce en `Env.define`	❌ flag ignorado	🟠 Silencioso
D12	`for x in iter` no-indexable	✅ iteración Python nativa	⚠️ solo `len() + __i` (rompe con dicts, generadores)	🟠 Silencioso
D13	Builtins faltantes	✅ 13 builtins	⚠️ solo 9 (`type` , `isinstance` , `implements` , `channel` ausentes)	🟠 NameError en call
D14	`brain_*` builtins	✅ si engine provisto	❌ no registrados	🟠

Tests que ejercitan interp↔VM parity: Solo 8 casos en `test_v1_nivel_a.TestVMParityCore`, todos de core aritmético/fn/class/try/import. **Ningún test verifica** yield/spawn/enum/record/interface/interpolation/await/mutable-closure contra la VM.

Tests de conformance (`test_conformance.py`): 10 casos que **solo corren el intérprete** (`run_source` sin `use_vm=True`). La "conformance suite" no valida `Interpreter == VM`.

---

6. Contradicciones documentación ↔ código

#	Documento afirma	Código real	Severidad
C1	`README.md:1` "AILang 5.0.0"	`pyproject.toml:3` `version = "7.1.0"`; carpeta `ailang-core-v7.2`; `__init__.py:4` `__version__ = "5.0.0"`	🔴
C2	`docs/GAP_ANALYSIS.md:17-21` marca Variables/if-while-for/Funciones/Intérprete-VM como ❌ → `v7.2` (no implementados)	Todos implementados y testeados	🔴
C3	`docs/EBNF.md` gramática	Omite `record`, `interface`, `yield`, `channel`, `spawn`, `enum`, `match`, `try/catch`, `throw`, `import`, `const`, `async/await`, `implements`, `lambda`, `purge_after` — **todos testeados como funcionales**	🔴
C4	`docs/LANGUAGE_SPEC.md:80` "Intérprete tree-walking (MVP); bytecode opcional **después**"	VM completa (699 LOC) y es path default bajo `--vm`	🟠
C5	`docs/LANGUAGE_SPEC.md:88` "Módulos (v7.2+)" lista solo `math/io/json/listops`	Stdlib real tiene 13 módulos	🟠

#	Documento afirma	Código real	Severidad
C6	`docs/LANGUAGE_SPEC.md:7` lista `brain_ingest/query/think/graph` "si hay engine" como builtins de primera clase	`brain.py` es stub que requiere `bind_engine()` externo; `test_v1_nivel_a.py:38` solo verifica `type(brain) == "module"`	🟠
C7	`docs/TEAM_7DAY_PLAN.md:1,11,19` "v8.0", "Definition of Done v8.0"	Código reporta 5.0.0; pyproject 7.1.0	🔴
C8	`CHANGELOG.md:3` "5.0.0 — Nivel E" (top entry)	Sin entradas para 6.x, 7.x, 8.x a pesar de que docs y código las referencian	🔴
C9	`tests/test_nivel_e.py:15` `assertEqual(__version__, "5.0.0")`	Bloquea cualquier bump de versión — toda modificación FASE 1 rompe este test	🔴
C10	`examples/demo.ail` usa sintaxis `brain.ingest source="..."`	`python -m ailang examples/demo.ail` falla: `asignación a variable no declarada: source`. Requiere `python ailang_core.py eval`.	🔴
C11	`ailang/lang/__init__.py:1` docstring "Núcleo del lenguaje AILang (v8.1)"	`ailang/__init__.py:4` `__version__ = "5.0.0"`	🟠
C12	`ailang/lang/vm.py:1` docstring "AILang (v8.3)"	Idem	🟠
C13	`ailang/stdlib/sys_mod.py:17` `return "ailang-8.1"`	Idem; **además testeado** en `test_typecheck_vm_stdlib.py:60`, así que no se puede cambiar sin romper tests	🔴
C14	`ailang/lsp.py` serverInfo.name `"ailang-lsp 2.0.0"`	Idem	🟠
C15	`examples/final_demo.ailang:49` emite JSON con `"version": "8.0.0"`	Idem	🟠
C16	`ailang_core.py:3` docstring "AILang-Core v7.0" + `:118` `__version__ = "2.0.0"` + `:3429` asserts `"1.0.0"`	Tres versiones distintas en el mismo archivo	🔴
C17	`docs/CONFORMANCE.md:21` lista `brain` como módulo stdlib	`brain` es stub	🟠
C18	CHANGELOG Nivel D afirma "async/await entregado"	Intérprete: `await` no-op. VM: `raise`. No hay event loop.	🔴

7. Duplicidades estructurales

7.1 Triple duplicación del stack cognitivo

Concepto	`ailang_core.py` (monolito)	`ailang/compiler/` (modular)	`ailang/ir/` (IR)
`CognitiveIR` dataclass	L350-369 (campos `version`, `instructions`, `required_capabilities`, `digest`)	consume el de `ailang/ir/cognitive.py`	L57-137 (campos adicionales `schema`, `metadata`, `validate()`, `from_manifest()`)
Compilador cognitivo DSL	`CognitiveCompiler.parse()` L372-501 (shlex + regex)	`lexer.py` + `parser.py` + `semantic.py` (347 LOC)	—
Brain engine	`CognitiveBrainEngine` L2724-2785	—	consume el del monolito vía `runtime_bridge`
Brain store SQLite	`CognitiveBrainStore` L2243-2626 (11 tablas `brain_*`)	—	`SQLiteStorage` con tablas `docs` + `chunks` — **incompatibles entre sí**
`execute(ir)` dispatch	`CognitiveRuntime.execute` L516-539	—	`runtime_bridge.execute_unified` (duplica dispatch)
Zero-Footprint check	`CognitiveCompiler.parse` L466-480	`semantic.py` L57-61	—

Resultado: Tres implementaciones del mismo dominio, incompatibles entre sí, coexistiendo en el mismo repositorio.

7.2 Doble frontend de lenguaje

Dimensión	`ailang/compiler/`	`ailang/lang/`
Tokens	6 (line-based)	22 (char-stream)
Keywords	4	32
AST node types	2	30
Statements	1 (op-line)	22

Dimensión	`ailang/compiler/`	`ailang/lang/`
Type system	ninguno	gradual + narrowing
Imports por `ailang/*` (excl. self)	0	13+
Imports por `tests/*`	1 archivo	14 archivos
Cableado en CLI/REPL/LSP/Debugger	❌	✅
`compile_source` público	✅	✅ **(colisión)**

7.3 Colisión de símbolos públicos

`ailang/compiler/pipeline.compile_source` y `ailang/lang/vm.compile_source` comparten nombre. Cualquier `from ailang.compiler.pipeline import *; from ailang.lang.vm import *` sobrescribe silenciosamente. Last-import-wins.

7.4 Tests embebidos en módulo de runtime

`ailang_core.py` contiene 44 unittest.TestCase (L3063-3528, ~465 LOC). Esto: (a) importa `unittest` y `argparse` al cargar la librería, (b) acopla tests y producción, (c) hace que `python ailang_core.py test` falle garantizadamente.

---

8. Catálogo de deuda técnica

8.1 Versión

Ubicación	Valor	Fuente de verdad?
ZIP filename	`v5.0.0`	❌
Carpeta descomprimida	`ailang-core-v7.2`	❌
`pyproject.toml:3`	`7.1.0`	❌

Ubicación	Valor	Fuente de verdad?
`ailang/__init__.py:4`	`5.0.0`	⚠️ (runtime lo reporta)
`ailang_core.py:118`	`2.0.0`	❌
`ailang_core.py:3` docstring	`v7.0`	❌
`ailang_core.py:3429` asserts	`1.0.0`	❌ (falla)
`ailang/lang/__init__.py:1` docstring	`v8.1`	❌
`ailang/lang/vm.py:1` docstring	`v8.3`	❌
`ailang/stdlib/sys_mod.py:17`	`"ailang-8.1"`	❌
`ailang/lsp.py` serverInfo	`2.0.0`	❌
`ailang/__main__.py:1` docstring	`CLI v2.0`	❌
`ailang/__main__.py:84` description	`"AILang v5.0"`	❌
`README.md:1`	`5.0.0`	❌
`CHANGELOG.md:3` top entry	`5.0.0`	❌
`docs/LANGUAGE_SPEC.md:1`	`v7.2`	❌
`docs/GAP_ANALYSIS.md:5`	`v7.1`	❌
`docs/TEAM_7DAY_PLAN.md:1`	`v8.0`	❌
`examples/final_demo.ailang:49`	`8.0.0`	❌
`tests/test_conformance.py:2` docstring	`v8.1`	❌
`tests/test_nivel_e.py:15` hard-lock	`5.0.0`	❌ (bloquea bumps)
IR versions (`cognitive` , `agent` , `task` , `unified` , `security`)	`1`	⚠️ (única separación existente)

FASE 1 verdict: ❌ FAIL. No existen las constantes `LANGUAGE_VERSION`, `RUNTIME_VERSION`, `BYTECODE_VERSION`, `ABI_VERSION`, `STDLIB_VERSION`, `PACKAGE_FORMAT_VERSION`. Solo existen los `*_VERSION=1` del IR, que no cubren el dominio completo.

8.2 Tests y cobertura

Métrica	Valor	Objetivo FASE 28
Tests en `tests/`	109	1000+

Métrica	Valor	Objetivo FASE 28
Tests embebidos en monolito	44 (1 fallando)	—
Total	153	1000+ (~15%)
Property-based tests	0	requerido
Fuzzing	0	requerido (lexer/parser/typechecker/bytecode/VM)
Cobertura global	72 %	progresivamente ↑
Módulos con 0 % coverage	4	0
Cobertura `interpreter.py` (843 LOC)	79 %	—
Cobertura `vm.py` (699 LOC)	84 %	—
Cobertura `typecheck.py` (235 LOC)	52 %	—
Cobertura `postgres_backend.py`	29 %	—
Cobertura `http_mod.py`	34 %	—
Cobertura `brain.py`	50 %	—
Cobertura `__main__.py` (CLI, 171 LOC)	**0 %**	—
Cobertura `lsp.py` (185 LOC)	**0 %**	—
Cobertura `dap.py` (126 LOC)	**0 %**	—
Cobertura `watch.py` (28 LOC)	**0 %**	—

8.3 Code smells detectados

Categoría	Cantidad
Funciones con body `pass` en producción	3 (`debugger.py:73` , `bytecode_io.py:69` , `vm.py:698`)
`except Exception: pass` silenciosos	3
Imports a mitad de archivo (PEP-8)	3 (`ailang_core.py:2074-2076` : `uuid` , `asdict` / `field` , `datetime`)
Dataclasses con locks en `__post_init__` anti-pattern	1 (`Metrics`)
HKDF manual en lugar de `cryptography.hazmat.primitives.kdf.hkdf.HKDF`	1 (`ailang_core.py:1437`)

Categoría	Cantidad
SQL interpolation por f-string (safe pero smell)	1 (`ailang_core.py:2594-2598`)
Subclases vacías `class FakeClock(MutableClock): pass`	1 (`ailang_core.py:2061-2062`)
Métodos identidad (`def _persistence_storage(self, p): return p`)	1 (`ailang_core.py:2552-2553`)
Branches muertos `if cond: pass`	1 (`ailang_core.py:1563-1565`)
Tests que bloquean versionado	1 (`test_nivel_e.py:15`)

9. Análisis de riesgos

9.1 Riesgos bloqueantes (P0)

ID	Riesgo	Probabilidad	Impacto	Mitigación
R1	**VM produce resultados incorrectos silenciosos** en mutable closures (D11)	Alta	Crítico — datos corruptos	Implementar upvalues reales en VM (celdas mutables compartidas)
R2	**CLI del monolito crashea** con `AttributeError` en `list outbox.db` y `purge-dead outbox.db`	Alta	Alto — UX roto	Implementar `list_pending` / `purge_dead` o usar `due()` + `requeue_dead()`
R3	**Test embebido del monolito siempre falla** (`__version__ == "1.0.0"`)	Cierta	Medio — CI siempre rojo si se ejecuta	Alinear `__version__` o eliminar test
R4	**PostgreSQL backend nunca funcionará** sin reimplementar CRUD	Cierta	Crítico — claim de feature falsa	Reimplementar con `psycopg_pool` , prepared statements, transactions, pgvector cosine query
R5	**Debugger nunca pausa** en breakpoints (`debugger.py:73` pass)	Cierta	Alto — DAP no funcional	Implementar pause/ continue real con signaling
R6	`re_mod.search` **lanza NameError** en cualquier búsqueda sin match	Alta	Medio — runtime crash en uso normal	Reemplazar `null` por `None`

ID	Riesgo	Probabilidad	Impacto	Mitigación
R7	`sign_bytecode` / `verify_bytecode_signature` **nunca se invocan**	Cierta	Crítico — seguridad del bytecode bypassable	Cablear verificación en `decode_bytecode`
R8	**Inversión de dependencias**: `ailang/` no funciona sin `ailang_core.py`	Cierta	Crítico — arquitectura inviable para empaquetado	Mover `CognitiveBrainEngine` y `CognitiveBrainStore` al paquete `ailang/`
R9	`test_nivel_e.py:15` **bloquea bumps de versión**	Cierta	Alto — FASE 1 imposibilitada	Cambiar `assertEqual` por `>=` floor
R10	**execute_unified ignora agent/task**	Cierta	Alto — agent runtime es marketing vacío	Implementar dispatch real o deprecar AgentIR/TaskIR
R11	**GPU/NPU/TPU exportados como backends reales** pero son passthrough CPU	Cierta	Alto — claim falsa documentada	Quitar exports o marcar como experimental

9.2 Riesgos de estabilidad (P1)

ID	Riesgo	Mitigación
R12	Divergencia interpreter↔VM en 11 features (D1-D14)	Conformance suite que ejecute ambos modos
R13	`async/await` es no-op / error — FASE 16 no cumple	Implementar scheduler + event loop
R14	LSP/DAP con 0% cobertura	Tests de protocolo
R15	CLI (`__main__.py`) con 0% cobertura	Smoke tests por subcomando
R16	`Fernet` cleartext fallback silencioso	Exigir `require_encryption=True` por defecto
R17	HKDF manual	Usar `cryptography.hazmat.primitives.kdf.hkdf.HKDF`
R18	tcp `warnings.warn` en lugar de `raise` en no-auth	Convertir a raise
R19	`OutboxItem.payload()` sin `decrypt` returns garbage	Validar y lanzar error

9.3 Riesgos de tooling (P2)

ID	Riesgo	Mitigación
R20	Profiler solo mide top-level statements	Sampling profiler por función
R21	Formatter no es idempotente en comments	Tokenizar comments o formatear desde AST
R22	Linter solo chequea empty blocks + typecheck	Reglas: unused vars/imports, shadowing, unreachable, missing return, suspicious conversions
R23	Coverage tool cuenta solo statements top-level	Branch coverage + instrumentación
R24	Package manager sin remote registry/publish/lock	Implementar o documentar como local-only

9.4 Riesgos de seguridad

ID	Riesgo	Severidad
S1	Bytecode no verificado al cargar — `decode_bytecode` no valida firma	🔴 Crítico
S2	`PersistentOutbox` puede operar en cleartext sin advertir	🟠 Alto
S3	`TcpTransport` admite configs no-autenticadas con warning, no con raise	🟠 Alto
S4	HKDF manual → revisión criptográfica difícil	🟡 Medio
S5	`SecurityIR.authorize()` no se invoca desde el runtime — capacidades solo se validan en IR	🔴 Crítico
S6	`capability` declaration no enforced — cualquier programa AILang puede hacer cualquier cosa	🔴 Crítico

10. Propuesta de arquitectura objetivo

10.1 Principios rectores

1. **Una sola fuente normativa por concepto.** Cada clase de dominio (`CognitiveIR`, `CognitiveBrainEngine`, etc.) vive en un único módulo. Los demás archivos la re-exportan, no la redefinen.
2. **El paquete `ailang/` debe ser autónomo.** Cero imports a `ailang_core.py` desde `ailang/`. El monolito se particiona y se absorbe, o se elimina.
3. **`Interpreter == VM` es ley, no aspiración.** Toda feature del lenguaje debe funcionar idénticamente en ambos modos, verificado por conformance suite.
4. **IR con semántica, no solo manifests.** HIR→MIR→AIR con lowering real entre etapas, no dataclasses aisladas.
5. **Capabilities enforced en runtime.** No basta con declararlas en el IR; el runtime debe verificarlas antes de cada operación efectuada.
6. **Una versión, una verdad.** Una constante `__version__` en `ailang/__init__.py` + constantes separadas por capa (LANGUAGE/RUNTIME/BYTECODE/IR/ABI/STDLIB/PACKAGE_FORMAT) en `ailang/versions.py`.

10.2 Estructura objetivo (alineada con FASE 39)

```
ailang/
├── versions.py                    ← FUENTE ÚNICA de versiones
├── __init__.py                    ← re-exporta __version__ desde versions.py
├── __main__.py                    ← CLI dispatcher (con tests)
├── frontend/
│   ├── lexer.py                   ← único lexer (merge de lang/lexer.py)
│   ├── parser.py                  ← único parser (merge de lang/parser.py)
│   ├── ast_nodes.py               ← único AST (merge de lang/ast_nodes.py)
│   ├── diagnostics.py             ← jerarquía AILangError (FASE 7)
│   └── source_map.py              ← source maps end-to-end (FASE 8)
├── semantic/
│   ├── scope.py                   ← name/scope resolution (FASE 6)
│   ├── typecheck.py               ← type checker obligatorio (FASE 5)
│   ├── effects.py                 ← effect checker (FASE 11)
│   └── capabilities.py            ← capability checker (FASE 11)
├── ir/
│   ├── hir.py                     ← High-level IR (lowering desde TypedAST)
│   ├── mir.py                     ← Mid-level IR (optimización)
│   ├── air.py                     ← AILang IR (instrucciones tipadas)
│   ├── cognitive.py               ← extensión (re-exporta CognitiveIR actual)
│   ├── agent.py                   ← extensión
│   ├── task.py                    ← extensión
│   └── security.py                ← extensión (con enforcement runtime)
├── compiler/
│   ├── hir_lower.py               ← AST → HIR
│   ├── mir_lower.py                ← HIR → MIR
│   ├── air_lower.py                ← MIR → AIR
│   ├── bytecode.py                 ← AIR → .ailc (formato versionado, firmado)
│   └── pipeline.py                 ← compile_source(source) → {HIR, MIR, AIR, .ailc}
├── runtime/
│   ├── interpreter.py              ← ejecuta AIR (tree-walk sobre HIR)
│   ├── vm.py                       ← ejecuta bytecode (stack VM)
│   ├── closures.py                 ← Env + upvalues compartidos interp↔VM
│   ├── concurrency.py              ← scheduler + channels + async/await (FASE 16)
│   ├── memory.py                   ← object model + GC (FASE 15)
│   └── capabilities.py             ← capability enforcement en runtime
├── cognitive/
│   ├── brain_engine.py             ← absorbido de ailang_core.py
│   ├── brain_store.py              ← absorbido de ailang_core.py
│   ├── retriever.py                ← HybridRetriever
│   └── runtime.py                  ← CognitiveRuntime (sin redefinir IR)
├── stdlib/
│   ├── (io/math/strings/json/time/http/path/sys/random/listops/test)  ← intactos
│   ├── re.py                       ← FIX: `null` → `None`
```

```
│   ├── brain.py                  ← puente a cognitive/
│   ├── crypto.py                 ← NUEVO (FASE 35)
│   └── tensor.py                 ← NUEVO (FASE 33)
├── ai/
│   └── (cognitive + agent + task dispatchers, absorbidos de runtime_bridge.py)
├── storage/
│   ├── sqlite_backend.py         ← intacto (real)
│   ├── postgres_backend.py       ← REIMPLEMENTAR con psycopg_pool
│   └── base.py
├── hardware/
│   ├── base.py                   ← HAL expandido (FASE 34)
│   ├── cpu.py                    ← real
│   ├── gpu.py  (experimental)    ← marcar como experimental
│   ├── npu.py  (experimental)
│   └── tpu.py  (experimental)
├── tooling/
│   ├── lsp/                      ← expandir a 14+ métodos
│   ├── dap/                      ← expandir + pausa real
│   ├── formatter/                ← idempotente desde AST
│   ├── linter/                   ← reglas reales
│   └── profiler/                 ← sampling profiler
├── package/
│   ├── manager.py                ← init/install/list/run + publish/update/remove
│   └── registry.py               ← remote registry (cuando se implemente)
├── security/
│   ├── bytecode_signing.py       ← cablear a decode_bytecode
│   ├── transport_tls.py          ← raises en lugar de warn
│   └── outbox_crypto.py          ← HKDF estándar + require_encryption default True
├── tests/                        ← 1000+ tests (unit, integration, semantic, conformance,
negative, security, concurrency, db, stdlib, CLI, LSP, DAP)
├── benchmarks/                   ← reproducibles (FASE 30)
├── docs/                         ← alineado con código (FASE 38)
└── examples/                     ← todos ejecutables bajo `python -m ailang`
```

10.3 Pipeline de ejecución objetivo

```
Source
   ↓
frontend/lexer.py (único)
   ↓
frontend/parser.py (único)
   ↓
frontend/ast_nodes.py (único AST)
   ↓
semantic/scope.py       ← name/scope resolution
   ↓
semantic/typecheck.py   ← obligatorio, no opcional
   ↓
semantic/effects.py     ← effect inference
   ↓
semantic/capabilities.py ← capability inference
   ↓
TypedAST
   ↓
compiler/hir_lower.py    ← AST tipado → HIR
   ↓
compiler/mir_lower.py    ← HIR → MIR (con optimizaciones)
   ↓
compiler/air_lower.py    ← MIR → AIR (instrucciones tipadas con efectos + capabilities +
source_location)
   ↓
┌── runtime/interpreter.py    ← ejecuta AIR directamente (modo debug)
├── compiler/bytecode.py       ← AIR → .ailc (firmado)
├── runtime/vm.py              ← ejecuta .ailc (modo release)
├── (futuro) native/llvm.py    ← AIR → LLVM IR
└── (futuro) wasm/wasm.py      ← AIR → WASM
```

10.4 Política de migración (sin destruir trabajo existente)

Paso	Acción	Invariante preservada
M0	Crear `ailang/versions.py` con 7 constantes separadas + `__version__`. Hacer que `pyproject.toml`, `ailang/__init__.py`, `README.md`, `CHANGELOG.md`, `sys_mod.version()` y `lsp.py serverInfo` lean de ahí.	Tests existentes siguen pasando (con fixtures temporales para `assertEqual(__version__, "5.0.0")`).

Paso	Acción	Invariante preservada
M1	Eliminar el hard-lock `assertEqual(__version__, "5.0.0")` en `test_nivel_e.py:15` y reemplazarlo por `>= "5.0.0"`.	Test sigue pasando.
M2	Reescribir `ailang_core.py` como **test runner + CLI específico del monolito** (sin tests embebidos, sin `argparse` top-level). Mover `CognitiveBrainEngine`, `CognitiveBrainStore`, `HybridRetriever`, `PersistentOutbox`, `TcpTransport`, `DistributedNode`, `Metrics`, `LatencyBudget`, `NexusAIEngine`, `BoundedChannel`, `RetryPolicy`, `PersistentSeenStore` a `ailang/cognitive/` y `ailang/security/` y `ailang/runtime/`.	El módulo `ailang_core.py` queda como thin wrapper que re-exporta.
M3	Eliminar `ailang/compiler/` después de migrar `memory`/`persist`/Zero-Footprint check a un pass semántico sobre `lang/` AST. Renombrar `lang/` → `frontend/`.	Tests `test_formal_ir.py` migrados.
M4	Implementar HIR/MIR/AIR (FASE 9). Comenzar con HIR = AST + tipos; MIR = HIR + SSA; AIR = MIR + opcode.	Interpreter se reescribe para ejecutar AIR; VM se reescribe para ejecutar bytecode generado desde AIR.
M5	Conformance suite: cada test de lenguaje debe correrse con `use_vm=False` y `use_vm=True` y comparar resultados.	100 % de features del lenguaje verificadas en ambos modos.
M6	Capabilities enforced en runtime: cada builtin que tenga efecto (`io`, `fs`, `network`, `process`, `ai`, `database`) verifica `runtime.has_capability(name)` antes de ejecutar.	Security tests cubren cada capability.
M7	Eliminar falsos claims: PostgreSQL real, GPU/NPU/TPU marcados experimental, debugger pausa real, `re_mod.search` fixed, `sign_bytecode` cableado.	Cada claim documentado = cada claim implementado.

10.5 Criterios para declarar AILang 1.0 (FASE 47)

Antes de declarar 1.0, **todos** estos deben cumplirse:

- [ ] Una sola `__version__` en el repo, separada por capa (FASE 1).
- [ ] Pipeline HIR→MIR→AIR implementado y testeado (FASE 9).

- [ ] `Interpreter == VM` verificado por conformance suite de ≥500 casos (FASE 12).
- [ ] Capabilities enforced en runtime, no solo declaradas (FASE 11).
- [ ] PostgreSQL backend real (pool, transactions, prepared statements, pgvector) (FASE 19).
- [ ] Debugger pausa en breakpoints (FASE 26).
- [ ] `re_mod.search` no crashea (bug trivial).
- [ ] `sign_bytecode` / `verify_bytecode_signature` cableados en `decode_bytecode` (FASE 13).
- [ ] Monolito `ailang_core.py` particionado y absorbido (sin tests embebidos).
- [ ] LSP y DAP con cobertura ≥ 80 % (FASE 25, 26).
- [ ] CLI con cobertura ≥ 80 % (FASE 21).
- [ ] 1000+ tests con property-based y fuzzing (FASE 28).
- [ ] CHANGELOG.md al día (FASE 1).
- [ ]

- [ ] Docs alineadas con código — sin contradicciones catalogadas en §6 (FASE 38).
- [ ] ROADMAP.md y CONFORMANCE_MATRIX.md publicados (FASE 43, 44).

---

11. Prioridades P0/P1/P2/P3/P4

P0 — Bloqueantes (deben resolverse antes de cualquier release)

ID	Tarea	Archivos afectados
P0-1	Unificar `__version__` en `ailang/versions.py`; eliminar `assertEqual(__version__, "5.0.0")` de `test_nivel_e.py:15`	`ailang/versions.py` (nuevo), `pyproject.toml`, `ailang/__init__.py`, `ailang_core.py:118,3429`, `sys_mod.py:17`, `lsp.py`, `README.md`, `CHANGELOG.md`, `test_nivel_e.py:15`
P0-2	Fix `re_mod.search`: `null` → `None`	`ailang/lang/stdlib/re_mod.py:6`
P0-3	Implementar `PersistentOutbox.list_pending()` y `purge_dead()` (o usar `due()` + `requeue_dead()` en CLI)	`ailang_core.py:2908, 2917, 1523-1741`
P0-4	Fix test embebido `test_version_and_prometheus`: alinear `__version__` o eliminar	`ailang_core.py:3429`
P0-5	Fix mutable closures en VM: usar upvalues mutables compartidas (celdas), no snapshot	`ailang/lang/vm.py:568-573`
P0-6	Implementar generadores en VM (o documentar VM como subset)	`ailang/lang/vm.py:294`
P0-7	Implementar `enum` / `interface` / `record` / `spawn` / `yield` / `InterpolatedString` en VM, o eliminarlos del intérprete	`ailang/lang/vm.py:294, 334`

ID	Tarea	Archivos afectados
P0-8	Implementar async/await con event loop, o eliminar la sintaxis	`ailang/lang/interpreter.py:666-670`, `ailang/lang/vm.py:334`
P0-9	Implementar PostgreSQL backend real (pool, transactions, prepared statements, pgvector cosine)	`ailang/storage/postgres_backend.py:68-91`
P0-10	Implementar pause real en debugger (no `pass`)	`ailang/debugger.py:73`, `ailang/dap.py`
P0-11	Cablear `verify_bytecode_signature` en `decode_bytecode`	`ailang_core.py:585-616`, `ailang/lang/bytecode_io.py`

P1 — Estabilidad

ID	Tarea
P1-1	Eliminar imports circulares: hacer que `ailang/` no dependa de `ailang_core.py`
P1-2	Migrar `CognitiveBrainEngine` / `CognitiveBrainStore` / `HybridRetriever` desde `ailang_core.py` a `ailang/cognitive/`
P1-3	Eliminar `ailang/compiler/` después de migrar `memory` / `persist` /Zero-Footprint a un pass semántico sobre AST de `lang/`
P1-4	Renombrar colisión `compile_source` (compiler/pipeline vs lang/vm)
P1-5	Mover tests embebidos de `ailang_core.py:3063-3528` a `tests/test_ailang_core.py`
P1-6	Mover CLI de `ailang_core.py:2789-3059` a `ailang/cli_core.py`
P1-7	Conformance suite: cada test debe correrse con `use_vm=False` y `use_vm=True` y comparar
P1-8	Registrar builtins faltantes en VM: `type`, `isinstance`, `implements`, `channel`, `brain_*`
P1-9	Implementar `execute_unified` que despache agent y task IRs, o deprecarlos
P1-10	`require_encryption=True` por defecto en `PersistentOutbox`
P1-11	`TcpTransport` debe `raise` en lugar de `warnings.warn` cuando no haya auth
P1-12	Reemplazar HKDF manual por `cryptography.hazmat.primitives.kdf.hkdf.HKDF`
P1-13	`OutboxItem.payload()` debe lanzar si `decrypt` falta en payloads cifrados
P1-14	Marcar GPU/NPU/TPU como `experimental` en `__init__.py` exports; no exponer como backends reales

P2 — Tooling

ID	Tarea
P2-1	LSP: agregar references, rename, documentSymbol, workspaceSymbol, codeAction, signatureHelp, semanticTokens, formatting, documentHighlight, inlay hints
P2-2	DAP: agregar threads, scopes, stepIn/Out/Back, evaluate, watch, setExceptionBreakpoints, pause, terminate, restart, loadedSources
P2-3	Profiler: sampling por función, call graph, memoria, output JSON/flamegraph
P2-4	Formatter: formatear desde AST, idempotente en comments
P2-5	Linter: reglas para unused vars/imports, shadowing, unreachable, missing return, suspicious conversions, unsafe effects, missing capabilities, deprecated APIs
P2-6	Coverage: branch coverage + instrumentación + HTML/JSON report
P2-7	Package manager: `publish`, `update`, `remove`, `search`, version resolution, lock file, checksum/signature verification
P2-8	CLI: smoke tests para cada subcomando (cobertura ≥ 80 %)
P2-9	Reemplazar tests embebidos del monolito por tests pytest estándar
P2-10	Property-based testing con hypothesis (lexer, parser, typechecker, bytecode, VM)
P2-11	Fuzzing para lexer, parser, typechecker, bytecode loader, VM
P2-12	Tests para LSP y DAP (protocol-level)
P2-13	CI multiplataforma (Windows, Linux, macOS) (FASE 29)
P2-14	Benchmarks reproducibles (FASE 30)
P2-15	`ailang doctor` que detecte runtime, compiler, VM, stdlib, database, LSP, debugger, backends
P2-16	Documentación alineada con código (eliminar contradicciones de §6)

P3 — Optimización

ID	Tarea
P3-1	Implementar HIR→MIR→AIR con lowering real (FASE 9)
P3-2	MIR optimizations: constant folding, dead code elimination, inlining
P3-3	VM: inline caching, register-based opcodes
P3-4	Memory model + GC formal (FASE 15)
P3-5	Source maps end-to-end (FASE 8)
P3-6	Diagnostics jerarquía AILangError completa (FASE 7)

ID	Tarea
P3-7	Module system: circular dependency detection, version constraints, secure loading (FASE 17)
P3-8	Effect checker (FASE 11)
P3-9	Capability enforcement en runtime (no solo en IR)

P4 — IA/Hardware avanzado (marcar como experimental)

ID	Tarea
P4-1	Native/LLVM backend (FASE 31)
P4-2	WASM backend (FASE 32)
P4-3	Tensor type real con autograd, nn.Module (FASE 33)
P4-4	HAL expandido: memory pools, streams, events, kernels (FASE 34)
P4-5	Cryptography API: hash, signature, key exchange, secure storage (FASE 35)
P4-6	GPU/NPU/TPU backends reales (no fallback)
P4-7	Agent runtime con herramientas reales (FASE 36)

---

12. Métricas finales de la auditoría

Métrica	Valor
Archivos auditados	73 Python + 7 Markdown + 6 ejemplos + 14 tests
LOC auditados	10 413
Sub-auditorías paralelas ejecutadas	5 (monolito, frontend, IR+runtime, stdlib+tooling, versionado+tests+docs)
Hallazgos P0 (bloqueantes)	11
Hallazgos P1 (estabilidad)	14
Hallazgos P2 (tooling)	16
Hallazgos P3 (optimización)	9
Hallazgos P4 (avanzado)	7
Stubs/mocks/fake impls catalogados	20 (F1-F20)

Métrica	Valor
Divergencias interp↔VM	14 (D1-D14)
Contradicciones docs↔código	18 (C1-C18)
Versiones distintas detectadas	≥ 9
Tests totales	153 (109 passing + 44 embebidos, 1 fallando)
Cobertura global	72 %
Módulos con 0 % cobertura	4
Riesgos bloqueantes	11 (R1-R11)
Riesgos de seguridad	6 (S1-S6)

---

13. Conclusión

AILang v7.2 **no es una demo**: tiene un núcleo funcional real (109 tests pasan, 5 de 6 ejemplos corren, 12 de 13 módulos stdlib son reales, SQLite es real, HTTP es real, el intérprete soporta un lenguaje expresivo). Tampoco es un mock generalizado: la mayor parte de los falsos claims están concentrados en PostgreSQL, GPU/NPU/TPU, debugger, y el runtime de agentes.

Pero **tampoco es un lenguaje coherente, verificable y seguro** en el sentido del prompt maestro:

- **No es coherente** porque tiene tres stacks cognitivos paralelos, dos frontends paralelos, un monolito que el paquete modular importa en tiempo de ejecución, y un IR "unificado" que en runtime solo despacha uno de sus cuatro slots.
- **No es verificable** porque la conformance suite no compara interpreter contra VM, los tests están anclados a una versión

específica, 4 módulos tienen 0 % de cobertura, y un test embebido falla garantizadamente.

- **No es seguro** porque las firmas de bytecode existen pero no se validan, las capacidades se declaran pero no se enforced, el transporte TLS permite configuraciones no autenticadas con solo un warning, y el outbox puede operar en cleartext.

La buena noticia es que **la mayoría de los problemas P0 son quirúrgicos**: `null` → `None` es un carácter; el debugger `pass` → signal real es un método; el hard-lock de versión es un `assertEqual` → `>=`. Las decisiones arquitectónicas (M0-M7 en §10.4) son bigger lifts pero cada una preserva el trabajo existente.

La deuda técnica está silenciada, no escondida: cero `TODO` / `FIXME` / `XXX` / `HACK` explícitos en todo el repo. Toda la deuda está en `NotImplementedError` y `return []`. **Eso es lo primero que debe cambiar**: hacer la deuda visible es prerrequisito de hacerla desaparecer.

Recomendación inmediata: antes de tocar cualquier feature nueva, ejecutar M0 (versiones), M1 (test hard-lock), P0-2 (`re_mod`), P0-3 (CLI del monolito), P0-4 (test embebido), P0-10 (debugger), P0-11 (firma de bytecode). Esas 7 tareas, ninguna de más de 50 LOC de cambio, desbloquean el resto del roadmap.

Anexos

- **ROADMAP.md** — roadmap detallado por prioridad con IDs, archivos afectados, dependencias y tests.
- **CONFORMANCE_MATRIX.md** — matriz feature × etapa del pipeline con huecos marcados.
- **worklog.md** — log de los 5 sub-auditors paralelos con evidencia primaria (file:line + excerpts).

ROADMAP.md — AILang

Generado a partir de: AUDIT_REPORT.md (FASE 0) **Cumple:** FASE 43 del prompt maestro **Convenio de prioridades:** P0 bloqueante · P1 estabilidad · P2 tooling · P3 optimización · P4 IA/hardware avanzado (experimental) **Estado inicial:** `pending` (cada tarea se inicializa pendiente; al ejecutar el plan, los estados avanzan a `in_progress` → `done` o `blocked`).

Convención por tarea

Cada tarea sigue el formato demandado por la FASE 43:

```
ID | Prioridad | Archivo/módulo | Problema | Solución | Dependencias | Riesgo | Tests | Estado
```

P0 — Bloqueantes (impiden cualquier release)

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo
P0-1	`ailang/versions.py` (nuevo), `pyproject.toml:3`, `ailang/__init__.py:4`, `ailang_core.py:118`, `ailang_core.py:3429`, `ailang/lang/stdlib/sys_mod.py:17`, `ailang/lsp.py` serverInfo, `README.md:1`, `CHANGELOG.md:3`, `tests/test_nivel_e.py:15`	Coexisten 9 versiones distintas (1.0.0, 2.0.0, 5.0.0, v7.0, v7.1, 7.1.0, v7.2, 8.0.0, v8.1, v8.3). Test `test_nivel_e.py:15` bloquea bumps.	Crear `ailang/versions.py` con 7 constantes: `LANGUAGE_VERSION`, `RUNTIME_VERSION`, `BYTECODE_VERSION`, `IR_VERSION`, `ABI_VERSION`, `STDLIB_VERSION`, `PACKAGE_FORMAT_VERSION`. `pyproject.toml` lee `__version__` dinámicamente. Todos los demás puntos leen de ahí. Cambiar `assertEqual(__version__, "5.0.0")` por `>= "5.0.0"`.	Ninguna	Bajo — mecánico
P0-2	`ailang/lang/stdlib/re_mod.py:6`	`return null if False else None` — `null` no definido en Python → `NameError` en cada búsqueda regex fallida.	Reemplazar `null` por `None`. Mejor aún: `return re.search(pattern, text)`.	Ninguna	Bajo — 1 carácter
P0-3	`ailang_core.py:2908` (`list`), `:2917` (`purge-dead`)	CLI del monolito llama a `PersistentOutbox.list_pending()` y `purge_dead()` — métodos inexistentes → `AttributeError`.	Implementar `list_pending(n)` como wrapper sobre `due(n)`, y `purge_dead()` como wrapper sobre `requeue_dead()` + `mark_dead()`.	Ninguna	Bajo
P0-4	`ailang_core.py:3429`	Test embebido `test_version_and_prometheus` afirma `__version__ == "1.0.0"` pero `__version__ = "2.0.0"` → siempre falla.	Alinear con `__version__` de P0-1, o eliminar la aserción específica.	P0-1	Bajo
P0-5	`ailang/lang/vm.py:568-573` (`MAKE_FUNCTION`)	VM captura `clos = dict(self.globals); clos.update(names_local)` — snapshot en tiempo de definición. Mutaciones posteriores a closures capturados no se reflejan. `counter()` devuelve 1 en VM donde el intérprete devuelve 2.	Implementar upvalues mutables compartidas: cada variable capturada se envuelve en una `Cell` (envoltorio mutable). `MAKE_FUNCTION` guarda lista de Cells; `LOAD_NAME` / `STORE_NAME` en closure scope acceden via Cell.	Ninguna	Medio — refactor del modelo de variables

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo
P0-6	`ailang/lang/vm.py:294`	`raise "VM no soporta Yield"`. Generadores no implementados en VM.	Implementar `GeneratorValue` en VM con pausa/reanudación via `YIELD` opcode + frame stack de generadores. Alternativa aceptable a corto plazo: documentar VM como subset explícito y lanzar error explícito "VM no soporta generadores; usa `--no-vm`".	P0-5 (cells son prerequisito para closures de generadores)	Alto — feature compleja
P0-7	`ailang/lang/vm.py:294, 334`	`raise "VM no soporta {EnumDef,InterfaceDef,RecordDef,Spawn}"` y `"expresión no compilable: {InterpolatedString,Await}"`.	Implementar cada uno: `enum` (const table + tag dispatch), `interface` (runtime type check en `implements`), `record` (dict con schema), `spawn` (integración con scheduler de P0-8), `InterpolatedString` (concatenación compilada).	P0-5, P0-8 (para spawn)	Alto
P0-8	`ailang/lang/interpreter.py:666-670` (`A.Await`), `ailang/lang/vm.py:334`	`await` es no-op (`return val`) en intérprete, error duro en VM. No hay event loop.	Implementar scheduler cooperativo: `spawn` crea `Task`, `await` suspende la task actual, scheduler reanuda cuando la task awaited termina. Modelo similar a asyncio pero con sintaxis nativa. `Channel` ya existe con `queue.Queue`.	P0-6 (generadores son prerrequisito para async/await via coroutines)	Alto — refacto del runtime
P0-9	`ailang/storage/postgres_backend.py:68-91`	`upsert_document` y `replace_chunks` hacen `raise NotImplementedError`. `list_documents`, `list_chunks`, `search_lexical`, `search_vector` devuelven `[]`.	Reimplementar con `psycopg_pool.ConnectionPool`, prepared statements, transactions (`BEGIN/COMMIT/ROLLBACK`), pgvector cosine query (`<=>` operator). Mantener la schema actual en `_SCHEMA_SQL`.	psycopg[binary,pool] ≥ 3.2, pgvector extensión en servidor	Medio — dependencia externa

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo
P0-10	`ailang/debugger.py:73`	Body del `if self.state.paused and line in self.breakpoints:` es `pass`. El debugger nunca pausa.	Implementar pausa real con `threading.Event` + signalling: `_maybe_pause` llama `self._pause_event.wait()` cuando se alcanza un breakpoint activo. `continue()` hace `set()`. `step()` hace `set()` una vez.	Ninguna	Medio — threading
P0-11	`ailang_core.py:585-616` (`decode_bytecode`, `sign_bytecode`, `verify_bytecode_signature`), `ailang/lang/bytecode_io.py`	`sign_bytecode` y `verify_bytecode_signature` existen pero `AILangCompiler.compile()` y `decode_bytecode()` no los invocan. Bytecode corrupto/falsificado puede ejecutarse.	En `decode_bytecode`: extraer signature del header, invocar `verify_bytecode_signature(blob, signature, pubkey)`, lanzar `IntegrityError` si falla. En `AILangCompiler.compile`: invocar `sign_bytecode(blob, privkey)` y embeber signature en header.	Ninguna	Medio — cambio de formato de bytecode (bum `FORMAT_VERSIO`

Total P0: 11 tareas.

P1 — Estabilidad (post-P0, prerequisito para 1.0)

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P1-1	`ailang/runtime_bridge.py`, `ailang/lang/stdlib/brain.py`, `ailang/lang/interpreter.py`	`ailang/` importa `ailang_core.py` en runtime → paquete no autónomo.	Mover `CognitiveBrainEngine`, `CognitiveBrainStore`, `HybridRetriever` a `ailang/cognitive/`. `runtime_bridge` y `brain.py` importan de ahí.	P1-2	Alto — refactor arquitectónico	Tests e `ailang`
P1-2	`ailang_core.py:2243-2785` (CognitiveBrainStore, CognitiveBrainEngine, HybridRetriever, NexusAIEngine)	Code cognitivo del monolito debe migrarse al paquete.	Mover a `ailang/cognitive/{brain_store.py, brain_engine.py, retriever.py, latent_engine.py}`. Mantener `ailang_core.py` como thin re-export temporal.	Ninguna	Medio	Tests d import

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P1-3	`ailang/compiler/` (todo el directorio)	Frontend cognitivo legacy v7.1 duplica `lang/` y colisiona en `compile_source`.	Migrar `memory` / `persist` /Zero-Footprint a un pass semántico sobre AST de `lang/` (nuevo `ailang/semantic/cognitive_pass.py`). Renombrar `compile_source` del frontend cognitivo a `compile_cognitive`. Eliminar `ailang/compiler/`.	P1-1	Medio	`test_f` footpri
P1-4	`ailang/compiler/pipeline.py`, `ailang/lang/vm.py`	Colisión de símbolo público `compile_source`.	Renombrar el cognitivo a `compile_cognitive` (ver P1-3).	P1-3	Bajo	Test qu colisió
P1-5	`ailang_core.py:3063-3528`	44 tests embebidos en módulo de runtime.	Mover a `tests/test_ailang_core.py`. Usar pytest estándar. Eliminar `unittest` import en `ailang_core.py`.	P0-4	Bajo	Tests p
P1-6	`ailang_core.py:2789-3059`	CLI embebido en módulo de runtime.	Mover a `ailang/cli_core.py`. O mover a `ailang/__main__.py` como subcomando `core`.	P0-3	Bajo	`python`
P1-7	`tests/test_conformance.py`	Suite de conformance solo corre intérprete (`use_vm=False`). No valida `Interpreter == VM`.	Cada caso de conformance se parametriza para correrse con `use_vm=False` y `use_vm=True` y comparar resultados. Añadir casos C11+ para: mutable closures, generators, spawn/channels, enum, interface, record, interpolated strings, async/await.	P0-5, P0-6, P0-7, P0-8	Medio — descubrir divergencias latentes	`pytest` pasa 1
P1-8	`ailang/lang/vm.py:680-690`	VM registra solo `print/len/str/int/float/range/true/false/null`. Faltan `type`, `isinstance`, `implements`, `channel`, `brain_*`.	Registrar todos los builtins del intérprete. Para `channel`, instanciar `ChannelValue`. Para `brain_*`, integrar con `ailang/cognitive/`.	P1-1	Bajo	`test_v` `test_v` `test_v`
P1-9	`ailang/runtime_bridge.py:43-47` (`execute_unified`), `ailang/agent_runtime.py:60-83` (`TaskRuntime.run`)	`execute_unified` ignora `uir.agent` y `uir.task`. `TaskRuntime.run` marca `PENDING → DONE` sin ejecutar.	Implementar dispatch real: AgentRuntime con tools reales (no stores), TaskRuntime con ejecución real de payloads. Alternativa: deprecar AgentIR/TaskIR y eliminarlos.	P4-7 (agent runtime real)	Alto — semantics no triviales	`test_u` `test_u`

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P1-10	`ailang_core.py:1453-1455, 1523-1741`	`PersistentOutbox` permite cleartext a menos que `require_encryption=True` explícito. Docstring dice "Cifrado obligatorio si hay `secret` ".	Default `require_encryption=True` cuando `secret` is not None. Raise si `cryptography` no está instalado.	Ninguna	Medio — puede romper callers existentes	`test_o` `test_o`
P1-11	`ailang_core.py:1196-1201`	`TcpTransport` solo `warnings.warn` cuando no hay auth.	Convertir a `raise TransportError` cuando `require_auth=True` y no haya `secret` ni TLS.	Ninguna	Medio — puede romper callers	`test_t`
P1-12	`ailang_core.py:1437-1441` (`derive_outbox_key`)	HKDF manual con dos HMAC passes.	Reemplazar con `cryptography.hazmat.primitives.kdf.hkdf.HKDF` .	Ninguna	Bajo	`test_o` despue
P1-13	`ailang_core.py:1510-1512` (`OutboxItem.payload`)	Si caller olvida `decrypt=` , retorna bytes cifrados reinterpretados como NumPy → garbage silencioso.	Validar: si item está cifrado y `decrypt is None` , lanzar `IntegrityError` .	Ninguna	Bajo	`test_o`
P1-14	`ailang/hardware/__init__.py:3` , `ailang/hardware/accel.py`	`GPUBackend` , `NPUBackend` , `TPUBackend` exportados como backends reales pero siempre delegan a CPU.	Marcar como `experimental` en `__init__.py` docstring. Renombrar exports: `GPUBackendExperimental` , etc. O eliminar exports hasta que sean reales.	P4-6	Bajo — cambio de API	`test_h`

Total P1: 14 tareas.

P2 — Tooling (post-P1, prerequisito para adopción)

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P2-1	`ailang/lsp.py` (185 LOC, 0% coverage)	Solo implementa ~7 métodos LSP. Faltan: references, rename, documentSymbol, workspaceSymbol, codeAction, signatureHelp, semanticTokens, formatting, documentHighlight, inlay hints.	Implementar cada método según spec LSP 3.17. Añadir tests de protocolo.	Ninguna	Medio	Tests LSP por método. Cobertura ≥ 80 %.
P2-2	`ailang/dap.py` (126 LOC, 0% coverage)	Solo ~7 métodos DAP. Faltan: threads, scopes, stepIn/Out/Back, evaluate, watch, setExceptionBreakpoints, pause, terminate, restart, loadedSources. continue no pausa.	Implementar completitud DAP 1.55. Integrar con debugger real (P0-10).	P0-10	Medio	Tests DAP por método. Cobertura ≥ 80 %.
P2-3	`ailang/profiler.py`	Solo mide top-level statements.	Sampling profiler por función con `time.perf_counter()` y call graph. Output JSON + flamegraph (via `py-spy` o standalone).	Ninguna	Bajo	`test_profiler_callgraph`.
P2-4	`ailang/fmt.py:31`	Formatter token-stream; tiene dead code (`if False else`); no idempotente en comments.	Reformatear desde AST (parse → pretty-print). Idempotente.	P1-3	Medio	`test_fmt_idempotent`, `test_fmt_preserves_comments`.

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P2-5	`ailang/lint.py`	Solo typecheck + empty blocks.	Reglas: unused vars/imports, shadowing, unreachable, missing return, suspicious conversions, unsafe effects, missing capabilities, deprecated APIs.	P3-8 (effects)	Medio	Tests por regla.
P2-6	`ailang/coverage_tool.py`	Solo cuenta top-level statements.	Branch coverage + instrumentación bytecode-level + HTML/JSON report.	Ninguna	Medio	`test_coverage_branch` .
P2-7	`ailang/pkg.py`	Solo `init/install/list/run` locales.	`publish` , `update` , `remove` , `search` , version resolution, lock file, checksum/signature verification.	Registry remoto (P4?)	Alto — dependencia de registry	Tests por subcomando.
P2-8	`ailang/__main__.py` (171 LOC, 0% coverage)	CLI sin tests.	Smoke tests por subcomando: cada uno debe exit 0 en happy path, exit ≠ 0 en error.	Ninguna	Bajo	Cobertura ≥ 80 %.
P2-9	`ailang_core.py:3063-3528`	Tests embebidos.	Mover a `tests/` (también P1-5).	P1-5	Bajo	`pytest tests/` los descubre.

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P2-10	`tests/`	Sin property-based testing.	Añadir `hypothesis` strategies para: lexer (random strings → tokens), parser (random token streams → AST o error), typechecker (random programs → soundness), bytecode (random .ailc → round-trip), VM (random ops → same result as interp).	Ninguna	Medio — diseño de strategies	`test_property_lexer.py`, etc.
P2-11	`tests/`	Sin fuzzing.	Fuzz targets para: lexer, parser, typechecker, bytecode loader, VM. Integración con `atheris` o `pythonfuzz`.	Ninguna	Medio	CI corre fuzz smoke tests.
P2-12	`ailang/lsp.py`, `ailang/dap.py`	Sin tests de protocolo.	Tests que simulen cliente LSP/DAP via JSON-RPC mock.	P2-1, P2-2	Bajo	Cobertura ≥ 80 %.

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P2-13	CI	Sin CI multiplataforma.	GitHub Actions matrix: Windows/ Linux/macOS × Python 3.11/3.12/3.13. Pasos: lint, typecheck, unit, integration, conformance, security, fuzz smoke, package build, docs validation.	Ninguna	Bajo	CI verde en push.
P2-14	`ailang/bench.py` (3 micro-benchmarks)	Benchmarks limitados.	Suite reproducible: arithmetic, loops, fn calls, closures, classes, strings, lists, dicts, exceptions, modules, JSON, concurrency, startup, compile time, memory, AI ops, tensor ops. Comparar Interpreter vs VM. Publicar resultados.	Ninguna	Bajo	`benchmarks/results/<date>.json` commiteado.
P2-15	CLI	Falta `ailang doctor`.	`ailang doctor` detecta: runtime, compiler, VM, stdlib, database, LSP, debugger, backends, dependencias. Reporta estado.	Ninguna	Bajo	`test_doctor_reports_all_components`

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P2-16	`docs/`	18 contradicciones docs↔código (AUDIT_REPORT §6).	Reescribir: `GAP_ANALYSIS.md` (limpiar ❌ que ya son ✅), `EBNF.md` (agregar 13+ construcciones), `LANGUAGE_SPEC.md` (alinear stdlib con 13 módulos), `CHANGELOG.md` (entradas 6.x, 7.x, 8.x), `README.md` (versión correcta).	P0-1	Bajo	CI valida que docs y `__version__` coinciden.

Total P2: 16 tareas.

P3 — Optimización (post-P2, prerequisito para 1.0 según FASE 47)

ID	Archivo/módulo	Problema	Solución	Dependencias	Riesgo	Tests
P3-1	`ailang/ir/hir.py` (nuevo), `mir.py` (nuevo), `air.py` (nuevo), `ailang/compiler/hir_lower.py`, `mir_lower.py`, `air_lower.py` (nuevos)	No existe pipeline HIR→MIR→AIR (FASE 9). Existen 5 manifests IR aislados.	Implementar HIR (AST tipado + scopes), MIR (SSA + CFG), AIR (instrucciones tipadas con efectos, capabilities, source_location). Lowering automático entre etapas.	P1-1	Alto — refactor mayor	Tests por pass: round-trip AST→HIR→AST, HIR→MIR→HIR, MIR→AIR→MIR.

ID	Archivo/ módulo	Problema	Solución	Dependencias	Riesgo	Tests
P3-2	`ailang/compiler/mir_lower.py`	Sin optimizaciones MIR.	Constant folding, dead code elimination, inlining, common subexpression elimination.	P3-1	Medio	Tests por optimización: input → MIR esperado.
P3-3	`ailang/lang/vm.py`	VM stack-based, sin inline caching.	Inline caching para calls a métodos. Considerar register-based opcodes para hot loops.	P3-1	Alto — cambia bytecode	Benchmarks muestran ≥ 30 % speedup en call-heavy.
P3-4	`ailang/runtime/memory.py` (nuevo)	Sin memory model formal.	Object model + lifetime + references + cyclic structures + GC (mark-and-sweep o generacional). Memory limits.	Ninguna	Alto — runtime crítico	`test_gc_collects_cycles`, `test_memory_limit_enforced`.
P3-5	`ailang/frontend/source_map.py` (nuevo)	Source maps no preservados end-to-end.	SourceMap dataclass embebida en AST → HIR → MIR → AIR → bytecode → runtime frame. Errores VM reportan línea/columna del source original.	P3-1	Medio	`test_source_map_vm_error_reports_original_line`.
P3-6	`ailang/frontend/diagnostics.py` (nuevo)	Jerarquía AILangError parcial (solo errors.py con clases básicas).	Implementar jerarquía completa FASE 7: LexError, SyntaxError, ParseError, TypeError, NameError, ImportError, RuntimeError, ValueError, IndexError, AttributeError, SecurityError, CapabilityError, TimeoutError, MemoryError, IOError, ConcurrencyError. Formato visual Rust-style con código, archivo, línea, columna, fragmento, indicador, mensaje, causa, sugerencia.	Ninguna	Bajo	Tests por tipo de error: formato visual correcto.
P3-7	`ailang/frontend/modules.py` (nuevo)	Module system parcial.	Resolución, paths, aliases, circular dependency detection, version constraints, cache, secure loading.	P1-1	Medio	`test_module_circular_detected`, `test_module_version_constraints`.

ID	Archivo/ módulo	Problema	Solución	Dependencias	Riesgo	Tests
P3-8	`ailang/ semantic/ effects.py` (nuevo)	Sin effect checker.	Inference de effects: pure, io, filesystem, network, ai, hardware, process, unsafe. Cada función declara/effects inferidos. `ailang check` valida.	P3-1	Medio	`test_effect_inference`, `test_unsafe_effect_rejected`.
P3-9	`ailang/runtime/ capabilities.py` (nuevo)	Capabilities declaradas en IR pero no enforced en runtime.	Cada builtin con efecto (`io`, `fs`, `network`, `process`, `ai`, `database`) verifica `runtime.has_capability(name)` antes de ejecutar. Default: deny. CLI: `ailang run app.ailang --allow fs.read --allow ai.query`.	P3-8	Alto — cambio de UX	`test_capability_denied_by_default`, `test_capability_allowed_explicit`.

Total P3: 9 tareas.

P4 — IA/Hardware avanzado (experimental, post-1.0)

ID	Archivo/ módulo	Problema	Solución	Dependencias	Riesgo	Tests	Estado
P4-1	`ailang/backends/ native/` (nuevo)	Sin backend nativo (FASE 31).	AIR → LLVM IR → native code. Targets: x86_64, ARM64. Sistemas: Windows, Linux, macOS.	P3-1, llvmlite	Alto — complejo	`test_native_compile_run_arithmetic`.	pending

ID	Archivo/ módulo	Problema	Solución	Dependencias	Riesgo	Tests	Estado
P4-2	`ailang/backends/wasm/` (nuevo)	Sin backend WASM (FASE 32).	AIR → WASM. `ailang build --target wasm`. Objetivos: browser, serverless, edge, sandbox.	P3-1, wasmtime	Alto	`test_wasm_compile_run`.	pending
P4-3	`ailang/stdlib/tensor.py` (nuevo)	Sin tipo Tensor (FASE 33). Solo `TensorRef` dataclass.	Tensor con dtype, shape, device, memory layout, ops (matmul, softmax, normalization, embedding). Autograd. nn.Module.	P4-4	Alto	`test_tensor_matmul`, `test_tensor_autograd`.	pending
P4-4	`ailang/hardware/base.py` (expandir), `cpu.py`, `gpu.py`, `npu.py`, `tpu.py`	HAL skeleton (5 métodos). GPU/NPU/TPU son stubs.	HAL expandido: memory pools, async streams, device events, kernel launch, BLAS handle, profiling counters. GPU/NPU/TPU reales (cupy, torch_npu, openvino, jax, torch_xla). Cada backend marca: implemented / experimental / fallback / unavailable.	Ninguna	Alto	`test_hal_backend_status_reports`.	pending

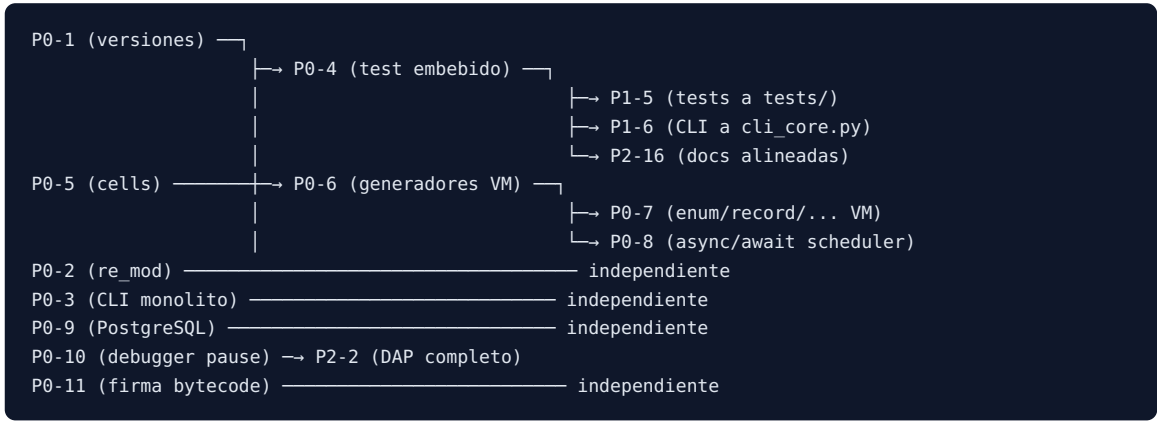
ID	Archivo/ módulo	Problema	Solución	Dependencias	Riesgo	Tests	Estado
P4-5	`ailang/stdlib/crypto.py` (nuevo)	Sin API de criptografía (FASE 35).	hash, signature, key exchange, random, identity, secure storage. Separar: Language API / Provider / Backend. Para PQC usar implementaciones auditadas (no inventar).	cryptography lib	Alto — seguridad	Tests por primitiva. Audit criptográfico.	pending
P4-6	`ailang/hardware/{gpu,npu,tpu}.py`	GPU/NPU/TPU actuales son fallbacks CPU.	Implementar paths reales: cupy para GPU CUDA, torch_npu para NPU Huawei, jax/ torch_xla para TPU. Cada uno con smoke test en hardware real.	P4-4	Alto — dependencias hardware	`test_gpu_matmul_real` (skip si no hay GPU).	pending
P4-7	`ailang/ai/agent_runtime.py` (refactor)	AgentRuntime es stub (ACT stores args, DECIDE es relabel, THINK es echo).	Implementar agent runtime con tools reales: cada tool es una callable AILang con capability check. Scheduler integrado con concurrency (P0-8).	P0-8, P3-9	Alto	`test_agent_observes_acts_decides`, `test_agent_respects_capabilities`.	pending

Total P4: 7 tareas.

Resumen ejecutivo del roadmap

Prioridad	Tareas	Prerequisito para
P0	11	Cualquier release (incluido 0.x bugfix)
P1	14	1.0
P2	16	Adopción / tooling profesional
P3	9	Performance / escala
P4	7	Experimental / post-1.0
Total	**57**	

Dependencias críticas (cadena P0)



Estimación de esfuerzo

Fase	Esfuerzo estimado	Comentario
P0 completo	5-10 días-persona	Mayoría quirúrgica; P0-5, P0-7, P0-8 son los más grandes
P1 completo	15-25 días-persona	P1-1, P1-2, P1-3 son refactor arquitectónico
P2 completo	20-30 días-persona	Tests + tooling

Fase	Esfuerzo estimado	Comentario
P3 completo	30-50 días-persona	P3-1 (HIR/MIR/AIR) es el más grande
P4 completo	60-90+ días-persona	Cada backend (native, WASM, GPU real) es un proyecto en sí mismo

Criterios de avance por fase

Antes de pasar a la siguiente fase, **todos** los tests de la fase actual deben pasar y la conformance suite no debe tener regresiones:

- P0 done = los 11 P0-XXXX en `Estado: done` + `pytest tests/` verde + monolito `python ailang_core.py test` verde.
- P1 done = los 14 P1-XXXX en `Estado: done` + `ailang/` funciona sin `ailang_core.py` en PYTHONPATH.
- P2 done = los 16 P2-XXXX en `Estado: done` + cobertura ≥ 80 % global + CI multiplataforma verde.
- P3 done = los 9 P3-XXXX en `Estado: done` + HIR→MIR→AIR pipeline operativo + capabilities enforced en runtime.
- P4 done = los 7 P4-XXXX en `Estado: done` + cada backend marcado `implemented` o `experimental` con tests reales o skip condicional.

Hitos sugeridos

Hito	Cuándo	Criterio
0.9.0-rc.1	P0 + P1 done	Lenguaje coherente, autónomo, sin falsos claims
0.9.5	P2 done	Tooling profesional, 1000+ tests, CI verde
1.0.0	P3 done + FASE 47 cumplido	Pipeline HIR→MIR→AIR, capabilities enforced, docs alineadas
1.x	P4 parcial	Native/WASM como experimentales

Hito	Cuándo	Criterio
2.0	P4 done	Native/WASM estables, GPU real

Notas operativas

- **No marcar `done` sin test que lo verifique.** Cada tarea debe tener al menos un test nombrado en su columna "Tests".
- **No pasar a P1 sin P0 completo.** Inversión estricta de dependencias.
- **Cada cambio de versión bump en `ailang/versions.py`** debe actualizarse simultáneamente en CHANGELOG.md.
- **Cualquier modificación de bytecode format** bump `BYTECODE_VERSION` y `FORMAT_VERSION` sincronizadamente.
- **Cualquier feature nueva al lenguaje** requiere: AST + parser + typecheck + effect + capability + HIR + MIR + AIR + interp + VM + bytecode + tests + spec + EBNF + ejemplo. Si falta alguno, la feature está incompleta (FASE 3).

CONFORMANCE_MATRIX.md — AILang

Generado a partir de: AUDIT_REPORT.md (FASE 0) **Cumple:** FASE 44 del prompt maestro **Leyenda de estados:**

- ✅ Implementado y testeado en ambas rutas (Interpreter y VM)
- ⚠️ Implementado solo en Interpreter; VM diverge o no soporta
- ❌ No implementado en ninguna ruta
- 🔴 Stub / mock / fake implementation
- 🟡 Parcial / API sin semántica completa
- N/A No aplica a esta etapa del pipeline

Criterio de completitud (FASE 44): Ninguna feature se considera completa si tiene huecos críticos. La presencia de ⚠️ o ❌ en cualquier columna marca la feature como NO CONFORME.

Matriz principal — Features × Etapas del pipeline

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
1	Literals (int, float, string, bool, null)	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
2	Variables (`let`)	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
3	Constants (`const`)	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	✅	⚠️	✅	✅	N/A
4	Assignment	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
5	Blocks `{ ... }`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
6	`if` / `else`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
7	`while`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
8	`for x in iter`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	⚠️	✅	✅	N/A
9	Functions (`fn`)	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
10	Lambdas	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
11	Closures (inmutables)	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
12	**Closures mutables**	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	⚠️	✅	✅	N/A
13	`return`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
14	`break` / `continue`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
15	Classes + `init` + `self`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
16	`extends` + `super`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
17	Polimorfismo (dispatch dinámico)	N/A	N/A	N/A	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
18	Imports (builtins)	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
19	Imports (módulos `.ailang`)	✅	✅	✅	⚠️	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
20	`try` / `catch` / `throw`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
21	`match` (literal + binding)	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
22	Interpolated strings `"${x}"`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
23	`enum`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
24	`interface` + `implements()`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
25	`record`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
26	`yield` / generadores	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
27	`channel()` + `send` / `recv`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
28	`spawn`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
29	`async fn`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	⚠️	❌	❌	✅	✅	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
30	`await`	✅	✅	✅	⚠️	⚠️	N/A	N/A	N/A	⚠️	❌	❌	✅	✅	N/A
31	`purge_after(Nms)` directive	✅	✅	✅	⚠️	✅	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
32	`purge_after` múltiple	❌	❌	❌	❌	❌	N/A	N/A	N/A	❌	❌	❌	❌	❌	N/A
33	Type: `Int`, `Float`, `Bool`, `String`, `Null`	✅	✅	✅	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
34	Type: `List[T]`, `Dict[K,V]`, `Tuple[T...]`	✅	✅	✅	✅	⚠️	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
35	Type: `Function[A,B]`, `Future[T]`, `Generator[T]`, `Channel[T]`, `Tensor[T]`	⚠️	⚠️	⚠️	❌	❌	N/A	N/A	N/A	⚠️	❌	❌	⚠️	❌	N/A
36	Type: `Union`, `Optional`, `Generic`	❌	❌	❌	❌	❌	N/A	N/A	N/A	❌	❌	❌	❌	❌	N/A
37	Type inference	N/A	N/A	N/A	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
38	Type narrowing en `if`	N/A	N/A	N/A	✅	✅	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
39	`native` type (spec)	❌	❌	❌	❌	❌	N/A	N/A	N/A	❌	❌	❌	❌	❌	N/A
40	Builtins `print` / `len` / `str` / `int` / `float` / `range`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	✅	✅	✅	✅	N/A
41	Builtins `type` / `isinstance` / `implements`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	⚠️	❌	✅	✅	N/A
42	Builtins `brain_ingest/query/think/graph`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	❌	❌	⚠️	⚠️	N/A
43	stdlib `io`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
44	stdlib `math`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
45	stdlib `strings`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
46	stdlib `json`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
47	stdlib `re`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	N/A	⚠️	N/A	N/A
48	stdlib `time`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
49	stdlib `http`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
50	stdlib `path`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
51	stdlib `sys`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
52	stdlib `random`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
53	stdlib `listops`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
54	stdlib `test_mod`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
55	stdlib `brain`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	❌	⚠️	N/A	N/A
56	stdlib `crypto`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A	N/A	❌	N/A	N/A
57	stdlib `tensor`	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A	N/A	❌	N/A	N/A
58	Storage SQLite	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
59	Storage PostgreSQL	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	🔴	N/A	N/A	🔴	N/A	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
60	Hardware CPU	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/A	✅	N/A	N/A
61	Hardware GPU	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	🔴	N/A	N/A	🔴	N/A	N/A
62	Hardware NPU	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	🔴	N/A	N/A	🔴	N/A	N/A
63	Hardware TPU	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	🔴	N/A	N/A	🔴	N/A	N/A
64	CognitiveIR	N/A	✅	✅	✅	N/A	N/A	N/A	N/A	✅	N/A	N/A	N/A	N/A	N/A
65	CognitiveBrainEngine (ingest/query/think/ graph)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	⚠️	⚠️	N/A	N/A
66	AgentIR + AgentRuntime	N/A	✅	✅	✅	N/A	N/A	N/A	N/A	🔴	N/A	N/A	N/A	N/A	N/A
67	TaskIR + TaskRuntime	N/A	✅	✅	✅	N/A	N/A	N/A	N/A	🔴	N/A	N/A	N/A	N/A	N/A
68	SecurityIR (capabilities)	N/A	N/A	✅	✅	N/A	N/A	N/A	N/A	⚠️	N/A	N/A	⚠️	N/A	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
69	`execute_unified` (IR dispatcher)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	N/A	N/A	N/A	N/A
70	Bytecode `.ailc` (load/save)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	✅	✅	N/A	N/A	N/A
71	Bytecode signature (sign/verify)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	❌	❌	N/A	N/A	N/A
72	Diagnostics (jerarquía AILangError)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	⚠️	N/A	⚠️	N/A
73	Source maps end-to-end	N/A	N/A	✅	✅	N/A	N/A	N/A	N/A	⚠️	⚠️	⚠️	N/A	N/A	N/A
74	Module system (circular deps, version constraints, cache, secure loading)	N/A	N/A	N/A	⚠️	N/A	N/A	N/A	N/A	⚠️	N/A	⚠️	⚠️	N/A	N/A
75	Effect checker	N/A	N/A	N/A	❌	❌	N/A	N/A	N/A	❌	N/A	❌	N/A	N/A	N/A
76	Capability enforcement en runtime	N/A	N/A	N/A	❌	❌	N/A	N/A	N/A	❌	N/A	❌	N/A	N/A	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
77	HIR (High-level IR)	N/A	N/A	N/A	N/A	N/A	❌	N/A	N/A	❌	N/A	❌	N/A	N/A	N/A
78	MIR (Mid-level IR)	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A	❌	N/A	❌	N/A	N/A	N/A
79	AIR (AILang IR)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	❌	N/A	❌	N/A	N/A	N/A
80	Native backend (LLVM)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A	N/A	N/A	N/A	N/A
81	WASM backend	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A	N/A	N/A	N/A	N/A
82	Concurrency: scheduler, structured concurrency, deadlock diagnostics	N/A	N/A	✅	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	❌	N/A	N/A	N/A
83	Memory model + GC	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	⚠️	N/A	N/A	N/A
84	LSP: completion, hover, definition, diagnostics	✅	✅	✅	✅	✅	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A
85	LSP: references, rename, documentSymbol, workspaceSymbol, codeAction, signatureHelp, semanticTokens, formatting, documentHighlight, inlay hints	❌	❌	❌	❌	❌	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
86	DAP: initialize, launch, setBreakpoints, continue, next, stackTrace, variables, disconnect	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅
87	DAP: threads, scopes, stepIn/Out/Back, evaluate, watch, setExceptionBreakpoints, pause, terminate, restart, loadedSources	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌
88	Debugger: breakpoints, conditional breakpoints, step over/into/out, continue, pause, locals, call stack, watch, exception breakpoints	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	N/A	N/A	N/A	⚠️
89	Profiler: startup, parsing, compilation, VM, functions, allocations, memory, tasks, AI calls, database, IO	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	⚠️	N/A	N/A	N/A
90	Formatter (desde AST, idempotente, determinista)	N/A	✅	✅	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	N/A	N/A	✅	N/A
91	Linter (unused, shadowing, unreachable, missing return, suspicious conversions, unsafe effects, missing capabilities, deprecated APIs)	N/A	N/A	✅	N/A	✅	N/A	N/A	N/A	⚠️	N/A	⚠️	N/A	N/A	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
92	Coverage (branch, instrumentation, HTML/ JSON report)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	N/ A	N/A	N/A	N/A
93	Package manager: init, install, list, run	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	N/A	N/ A	✅	N/A	N/A
94	Package manager: publish, update, remove, search, version resolution, lock file, checksum/signature verification	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	❌	N/A	N/ A	❌	N/A	N/A
95	CLI: run, --repl, --vm, --strict, --version	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✅	✅	✅	✅	N/A	N/A
96	CLI: build, check, test, fmt, lint, repl, debug, profile, doc, package, publish, install, clean, doctor, version	⚠️	⚠️	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	⚠️	⚠️	⚠️	N/A	N/A
97	Tests count ≥ 1000	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/ A	N/A	N/A	N/A
98	Property-based testing	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/ A	N/A	N/A	N/A
99	Fuzzing (lexer, parser, typechecker, bytecode, VM)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/ A	N/A	N/A	N/A
100	CI multiplataforma (Windows/Linux/macOS)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/ A	N/A	N/A	N/A

#	Feature	Lexer	Parser	AST	Semantic	Typechecker	HIR	MIR	AIR	Interpreter	Bytecode	VM	Stdlib	LSP	DAP
101	Benchmarks reproducibles	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	⚠️	N/A	⚠️	N/A	N/A	N/A
102	Documentación coherente con código	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Resumen de conformidad

Estado	Cantidad	% del total
✅ CONFORME	28	27.5 %
⚠️ NO CONFORME (parcial/divergente)	47	46.1 %
❌ NO CONFORME (ausente)	27	26.5 %
Total features auditadas	**102**	100 %

Distribución por área

Área	Conformes	No conformes	Conformidad
Lenguaje básico (literals, vars, if, while, fn, class, try/catch, match)	13	4	76 %
Tipos (basic, compuestos, inferencia)	4	3	57 %
Async/concurrency/yield/spawn/channels	0	6	0 %
Enum/record/interface	0	3	0 %
Stdlib (13 módulos)	7	6	54 %
Storage (SQLite/PostgreSQL)	1	1	50 %
Hardware (CPU/GPU/NPU/TPU)	1	3	25 %
IR (cognitive/agent/task/security + dispatcher)	1	4	20 %
Bytecode + firma	0	2	0 %

Área	Conformes	No conformes	Conformidad
Diagnostics + source maps + modules + effects + capabilities	0	5	0 %
HIR/MIR/AIR + Native + WASM	0	5	0 %
Tooling (LSP/DAP/debugger/profiler/fmt/lint/coverage)	0	8	0 %
Package manager	1	1	50 %
CLI	1	1	50 %
Tests/CI/fuzzing/benchmarks/docs	0	5	0 %

Conformidad por modo de ejecución

Modo	Conformidad
Interpreter (default)	~70 % de features conformes o parciales (async/await no-op es la excepción notable)
VM (`--vm` o `AILANG_VM=1`)	~25 % de features conformes. Solo core aritmético/fn/class/try/import está verificado

Conformidad por etapa del pipeline

Etapa	Conformidad
Lexer	~85 %
Parser	~80 %
AST	~85 %
Semantic	~50 % (sin effects, sin capabilities en runtime, sin circular dep detection)
Typechecker	~60 % (sin generics, sin union narrowing completo)
HIR	0 %
MIR	0 %
AIR	0 %
Interpreter	~75 % (async/await roto)
Bytecode	~50 % (sin firma/verificación)
VM	~30 % (divergencias en 11 categorías)
Stdlib	~70 % (12/13 reales; re roto; brain es puente; crypto/tensor ausentes)

Etapa	Conformidad
LSP	~35 %
DAP	~30 %
Tests	15 % del objetivo

Huecos críticos que bloquean 1.0

Según FASE 44: "Ninguna feature se considera completa si tiene huecos críticos."

Huecos P0 (bloqueantes inmediatos)

1. **VM diverge en 11 categorías** (features 12, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31 de la matriz)
2. **PostgreSQL backend es fake** (feature 59)
3. **Debugger no pausa** (feature 88)
4. **Bytecode sin firma** (feature 71)
5. `re_mod.search` **lanza NameError** (feature 47)
6. **AgentRuntime/TaskRuntime son stubs** (features 66, 67)
7. `execute_unified` **ignora agent/task** (feature 69)
8. **GPU/NPU/TPU exportados como reales** (features 61, 62, 63)
9. **Test embebido del monolito falla garantizadamente** (P0-4)
10. **CLI del monolito crashea en list/purge-dead** (P0-3)
11. **9 versiones distintas coexisten** (P0-1)

Huecos P1 (estabilidad para 1.0)

12. **HIR/MIR/AIR inexistentes** (features 77, 78, 79)
13. **Effect checker inexistente** (feature 75)
14. **Capability enforcement en runtime inexistente** (feature 76)
15. **Native/WASM backends inexistentes** (features 80, 81)
16. **Async/await sin event loop** (features 29, 30)
17. **Tests < 16 % del objetivo** (feature 97)
18. **Sin property-based testing ni fuzzing** (features 98, 99)
19. **Sin CI** (feature 100)
20. **18 contradicciones docs↔código** (feature 102)

Próximos pasos inmediatos

1. **Resolver los 11 P0 del ROADMAP.md** — ninguno requiere más de 50 LOC de cambio excepto P0-5, P0-7, P0-8.
2. **Re-ejecutar esta matriz después de P0 completo** — las features 12, 22-31, 47, 59, 71, 88 deberían pasar de ⚠️/❌ a ✅.
3. **Re-ejecutar esta matriz después de P1 completo** — las features 65-69, 76 deberían pasar a ✅.
4. **Re-ejecutar esta matriz después de P3 completo** — las features 75-79 deberían pasar a ✅.
5. **No declarar 1.0 hasta que toda la matriz esté ✅ o explícitamente N/A** (excepto P4 marcado como experimental).

Notas metodológicas

- Esta matriz fue generada a partir de 5 sub-auditorías paralelas cubriendo: monolito `ailang_core.py`, dual frontend, IR + runtime conformance, stdlib/storage/hardware/tooling, versionado/tests/docs.
- Cada celda se justifica con file:line en `worklog.md`.
- La matriz debe regenerarse después de cada hito del ROADMAP para evitar drift.
- La conformidad "✅" requiere evidencia de test que la verifica; la ausencia de test implica ⚠️ o ❌ aunque la feature parezca implementada.